Practical Lab: Access Control Models

DAC, MAC, RBAC and ABAC

1 Introduction to Access Control

Access control is a security mechanism that determines who can access a resource and which operations they are allowed to perform.

Four major models are studied in this lab :

Model	Access based on
DAC	Resource ownership
MAC	Central security policy
RBAC	User roles
ABAC	Attributes and conditions

2 Discretionary Access Control (DAC)

2.1 Understanding DAC

In Discretionary Access Control (DAC), the owner of a resource determines who can access it and what permissions they have.

In Linux, each file has an owner, a group and three permission categories :

Permission	Symbol	Value
Read	r	4
Write	w	2
Execute	x	1

Permissions are assigned to :

— Owner : the file's owner.
— Group : members of the associated group.
— Others : all remaining users.

2.2 Guided experiment

Create a working directory and a file :

```
mkdir ~/DAC_Lab
cd ~/DAC_Lab
touch report.txt
ls -l report.txt
```

Modify its permissions :

```
chmod 740 report.txt
ls -l report.txt
```

The permission `740` means :

— Owner : read, write and execute.

— Group : read only.
— Others : no permissions.

Change the owner and group :

```
sudo groupadd accounting
sudo chown root:accounting report.txt
ls -l report.txt
```

Challenge 1 : Confidential Documents

A company stores sensitive financial documents in a directory named `/company/finance`.
The access policy is :

— The owner can read and modify documents.
— Members of the accounting group can read documents.
— Other users must have no access.

Tasks :

1. Create the directory and an example document.
2. Create the accounting group.
3. Assign the appropriate ownership.
4. Configure the permissions.
5. Verify the final configuration.
6. Suppose the directory has permission `750`, but a file inside it has permission `600`. Can a member of the accounting group read the file ? Justify your answer.

3 Mandatory Access Control (MAC)

3.1 Understanding MAC

Mandatory Access Control (MAC) enforces access according to a centrally defined security policy.
Unlike DAC, resource owners cannot freely modify the security policy.
Consider a military organization with three classification levels :

Classification	Description
Public	Accessible to everyone
Confidential	Restricted information
Secret	Highly sensitive information

Users receive security clearances. Access is determined by the security policy rather than file ownership.

3.2 Bell-LaPadula model

The Bell-LaPadula model aims to preserve confidentiality through two main rules :

— **No Read Up :** A user cannot read information above their security clearance.
— **No Write Down :** A user cannot write information into a lower security classification.

3.3 Practical exploration : SELinux

SELinux implements MAC using security labels and centrally enforced policies.
Check its status :

```
sestatus
```

Display security contexts :

```
ls -Z
```

Challenge 2 : Military Information System

A military organization has the following users :

User	Clearance
Alice	Secret
Bob	Confidential
Charlie	Public

The organization stores three documents : `Document_A` (Secret), `Document_B` (Confidential) and `Document_C` (Public).

Tasks :

1. Construct the access matrix indicating which documents each user can read.
2. Bob attempts to read Document_A. Should access be granted ?
3. Alice attempts to write information into Document_C. Is this operation allowed ?
4. Explain how Bell-LaPadula prevents information leakage.
5. A user has sufficient clearance to read a document, but the document owner explicitly denies access using ordinary Linux permissions. Is access necessarily granted under MAC ? Explain the difference between DAC and MAC enforcement.

4 Role-Based Access Control (RBAC)

4.1 Understanding RBAC

Role-Based Access Control assigns permissions according to organizational roles rather than directly to individual users.

For example :

Role	Permissions
Administrator	Full access
Developer	Read and write
Viewer	Read only

4.2 Guided experiment

Linux groups can simulate organizational roles.

Create the roles :

```
sudo groupadd administrators
sudo groupadd developers
sudo groupadd viewers
```

Create three users :

```
sudo adduser alice
sudo adduser bob
sudo adduser charlie
```

Assign roles :

```
sudo usermod -aG developers alice
sudo usermod -aG viewers bob
sudo usermod -aG administrators charlie
```

Verify membership :

```
groups alice
groups bob
groups charlie
```

Create organizational directories :

```
sudo mkdir -p /company/source
sudo mkdir -p /company/reports

sudo chown root:developers /company/source
sudo chmod 770 /company/source

sudo chown root:viewers /company/reports
sudo chmod 750 /company/reports
```

Challenge 3 : Enterprise Access Management

A company defines the following access policy :

Role	Source code	Reports
Developer	Read / Write	Read
Viewer	No access	Read
Administrator	Full access	Full access

Tasks :

1. Create the required groups and assign users.
2. Configure permissions for both directories.
3. Identify whether the previous configuration satisfies the complete access policy.
4. Correct any missing permissions.

5 Attribute-Based Access Control (ABAC)

5.1 Understanding ABAC

Attribute-Based Access Control makes authorization decisions based on attributes associated with :

— **Subject :** User characteristics (department, clearance).
— **Object :** Resource characteristics (classification).
— **Action :** Requested operation (read, write).
— **Environment :** Context (time, location).

Example policy :

```
IF
  user.department == document.department
  AND user.clearance >= document.classification
  AND 08:00 <= current_time <= 18:00

THEN GRANT ACCESS
ELSE DENY ACCESS
```

Challenge 4 : Banking Access Policy

A bank applies the following access policy :

— The employee's department must match the document's department.
— The employee's clearance must be sufficient.

— Access is permitted only between 08 :00 and 18 :00.

Evaluate the following access requests :

User	Department	Clearance	Document	Time
Alice	Finance	Secret	Finance / Secret	10 :00
Bob	HR	Confidential	Finance / Confidential	11 :00
Charlie	Finance	Public	Finance / Secret	14 :00
David	Finance	Secret	Finance / Secret	22 :00

Tasks :

1. Determine whether each request is granted or denied.
2. Identify the attribute responsible for each denial.
3. Alice requests access to an HR document. What happens ?
4. David requests access at 17 :00. Does the decision change ?
5. Explain why ABAC is more flexible than RBAC.

Extra Challenge – PortSwigger Access Control

Objective : Apply access-control concepts to real-world web security scenarios by identifying and exploiting access-control vulnerabilities in a controlled environment.

Platform : PortSwigger Web Security Academy

Resource : https ://portswigger.net/web-security/access-control